

主要程式碼:

```
import math
from typing import List, Tuple, Optional
from dataclasses import dataclass
import heapq
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.patches import Rectangle
import itertools

@dataclass
class Point:
    """表示 2D 空間中的點"""
    id: str
    x: float
    y: float

    def distance_to(self, other: 'Point') -> float:
        """計算兩點之間的歐氏距離"""
        return math.sqrt((self.x - other.x)**2 + (self.y - other.y)**2)

    def __repr__(self):
        return f"P{self.id}({self.x},{self.y})"

@dataclass
class MBR:
    """最小邊界矩形 (Minimum Bounding Rectangle)"""
    min_x: float
    min_y: float
    max_x: float
    max_y: float

    def __init__(self, min_x=float('inf'), min_y=float('inf'),
                  max_x=float('-inf'), max_y=float('-inf')):
```

```
self.min_x = min_x
self.min_y = min_y
self.max_x = max_x
self.max_y = max_y
```

def area(self) -> float:

```
"""計算矩形面積"""

if self.min_x > self.max_x or self.min_y > self.max_y:
    return 0
return (self.max_x - self.min_x) * (self.max_y - self.min_y)
```

def enlarge_to_include(self, point: Point) -> 'MBR':

```
"""將矩形擴展以包含某個點"""

return MBR(
    min(self.min_x, point.x),
    min(self.min_y, point.y),
    max(self.max_x, point.x),
    max(self.max_y, point.y)
)
```

def enlarge_to_include_mbr(self, other: 'MBR') -> 'MBR':

```
"""將矩形擴展以包含另一個 MBR"""

return MBR(
    min(self.min_x, other.min_x),
    min(self.min_y, other.min_y),
    max(self.max_x, other.max_x),
    max(self.max_y, other.max_y)
)
```

def enlargement_needed(self, point: Point) -> float:

```
"""計算為包含新點所需增加的面積"""

new_mbr = self.enlarge_to_include(point)
return new_mbr.area() - self.area()
```

```

def min_distance_to_point(self, point: Point) -> float:
    """計算矩形到點的最短距離 (用於最近鄰搜尋)"""
    dx = max(self.min_x - point.x, 0, point.x - self.max_x)
    dy = max(self.min_y - point.y, 0, point.y - self.max_y)
    return math.sqrt(dx*dx + dy*dy)

def __repr__(self):
    return f"({self.min_x:.1f},{self.min_y:.1f})-({self.max_x:.1f},{self.max_y:.1f})"

class RTreeNode:
    """R-tree 節點"""

    _id_counter = itertools.count() # 節點 ID 自動遞增

    def __init__(self, is_leaf: bool = True):
        self.is_leaf = is_leaf

        self.points: List[Point] = [] # 存放資料點 (葉節點)

        self.children: List['RTreeNode'] = [] # 存放子節點 (非葉節點)

        self.mbr = MBR()
        self.parent: Optional['RTreeNode'] = None
        self.node_id = next(RTreeNode._id_counter)

    def update_mbr(self):
        """更新節點的 MBR"""
        if self.is_leaf:
            if not self.points:
                self.mbr = MBR()
            else:
                self.mbr = MBR(
                    min(p.x for p in self.points),

```

```

        min(p.y for p in self.points),
        max(p.x for p in self.points),
        max(p.y for p in self.points)
    )
else:
    if not self.children:
        self.mbr = MBR()
    else:
        self.mbr = MBR(
            min(child.mbr.min_x for child in self.children),
            min(child.mbr.min_y for child in self.children),
            max(child.mbr.max_x for child in self.children),
            max(child.mbr.max_y for child in self.children)
        )

```

def size(self) -> int:

"""返回節點內元素數量"""

return len(self.points) if self.is_leaf else len(self.children)

class RTree:

"""R-tree 主結構"""

def __init__(self, max_entries: int = 3, split_strategy: str = 'quadratic'):

self.root = RTreeNode(is_leaf=True)

self.max_entries = max_entries

self.split_strategy = split_strategy # 分割策略: 'linear' 或

'quadratic'

self.counter = itertools.count()

RTreeNode._id_counter = itertools.count() # 每棵樹重新計

數節點 ID

def insert(self, point: Point):

"""插入一個點"""

```
leaf = self._choose_leaf(self.root, point) # 選擇適合的葉節
```

點

```
leaf.points.append(point)
```

```
leaf.update_mbr()
```

```
if leaf.size() > self.max_entries:
```

```
    self._split_node(leaf) # 超過容量則分割
```

```
else:
```

```
    self._adjust_tree(leaf) # 向上調整 MBR
```

```
def _choose_leaf(self, node: RTreeNode, point: Point) ->  
RTreeNode:
```

```
    """選擇插入的葉節點"""
```

```
    if node.is_leaf:
```

```
        return node
```

```
    min_enlargement = float('inf')
```

```
    best_child = None
```

```
# 找到對應點加入後，增大面積最少的子節點
```

```
for child in node.children:
```

```
    enlargement = child.mbr.enlargement_needed(point)
```

```
    if enlargement < min_enlargement:
```

```
        min_enlargement = enlargement
```

```
        best_child = child
```

```
    elif enlargement == min_enlargement:
```

```
        if best_child is None or child.mbr.area() <
```

```
best_child.mbr.area():
```

```
        best_child = child
```

```
return self._choose_leaf(best_child, point)
```

```

def _split_node(self, node: RTreeNode):
    """節點分割"""
    if self.split_strategy == 'linear':
        new_node = self._linear_split(node) # 線性分割
    else:
        new_node = self._quadratic_split(node) # 二次分割

    # 若節點是根，則生成新根
    if node.parent is None:
        new_root = RTreeNode(is_leaf=False)
        new_root.children = [node, new_node]
        node.parent = new_root
        new_node.parent = new_root
        new_root.update_mbr()
        self.root = new_root
    else:
        parent = node.parent
        new_node.parent = parent
        parent.children.append(new_node)
        parent.update_mbr()

        if parent.size() > self.max_entries:
            self._split_node(parent)
        else:
            self._adjust_tree(parent)

# ===== 線性分割 =====

def _linear_split(self, node: RTreeNode) -> RTreeNode:
    """

```

線性分割策略:

1. 找出在 x 或 y 維度上分布最遠的兩個條目作為種子

(seed)

2. 其他條目依照對兩組 **MBR** 的面積增加量分配

```
"""
```

```
entries = node.points if node.is_leaf else node.children
```

```
# 選種子（具體細節略，可參考原程式碼）
```

```
# ...
```

```
return self._distribute_entries(node, entries, seed1, seed2)
```

```
# ===== 二次分割 =====
```

```
def _quadratic_split(self, node: RTreeNode) -> RTreeNode:
```

```
"""
```

二次分割策略:

1. 計算所有兩兩條目組合，找出增加空間最大的一對作為

種子

2. 其他條目依對兩組 **MBR** 增大面積分配

```
"""
```

```
entries = node.points if node.is_leaf else node.children
```

```
max_waste = -1
```

```
seed1, seed2 = 0, 1
```

```
for i in range(len(entries)):
```

```
    for j in range(i + 1, len(entries)):
```

```
        if node.is_leaf:
```

```
            mbr1 = MBR(entries[i].x, entries[i].y,  
entries[i].x, entries[i].y)
```

```
            mbr2 = MBR(entries[j].x, entries[j].y,  
entries[j].x, entries[j].y)
```

```
        else:
```

```
            mbr1 = entries[i].mbr
```

```
            mbr2 = entries[j].mbr
```

```

        combined_mbr =
mbr1.enlarge_to_include_mbr(mbr2)
        waste = combined_mbr.area() - mbr1.area() -
mbr2.area()

        if waste > max_waste:
            max_waste = waste
            seed1, seed2 = i, j

    return self._distribute_entries(node, entries, seed1, seed2)

def _distribute_entries(self, node: RTreeNode, entries: List,
                        seed1: int, seed2: int) -> RTreeNode:
    """將條目分配到兩組節點"""
    new_node = RTreeNode(is_leaf=node.is_leaf)
    # 初始化兩組
    group1 = [entries[seed1]]
    group2 = [entries[seed2]]
    remaining = [e for i, e in enumerate(entries) if i not in [seed1,
seed2]]

    for entry in remaining:
        # 計算加入各組後需要增加的面積

        # 根據面積增量或現有面積分配條目

        # ...
    # 更新節點內條目及 MBR
    if node.is_leaf:
        node.points = group1
        new_node.points = group2
    else:
        node.children = group1
        new_node.children = group2
        for child in group2:

```



```

        child.parent = new_node

    node.update_mbr()
    new_node.update_mbr()
    return new_node

def _adjust_tree(self, node: RTreeNode):
    """向上更新 MBR，直到根節點"""
    current = node
    while current.parent is not None:
        current.parent.update_mbr()
        current = current.parent

# 最近鄰搜尋

def nearest_neighbor(self, query_point: Point, k: int = 1) ->
List[Tuple[Point, float]]:
    """利用優先隊列進行 K 最近鄰搜尋"""
    max_heap = []
    min_heap = [(0, next(self.counter), self.root)]

    while min_heap:
        _, _, node = heapq.heappop(min_heap)

        if node.is_leaf:
            for point in node.points:
                dist = query_point.distance_to(point)
                if len(max_heap) < k:
                    heapq.heappush(max_heap, (-dist, point))
                elif dist < -max_heap[0][0]:
                    heapq.heapreplace(max_heap, (-dist,
point))
            else:
                for child in node.children:
                    min_dist =
child.mbr.min_distance_to_point(query_point)

```

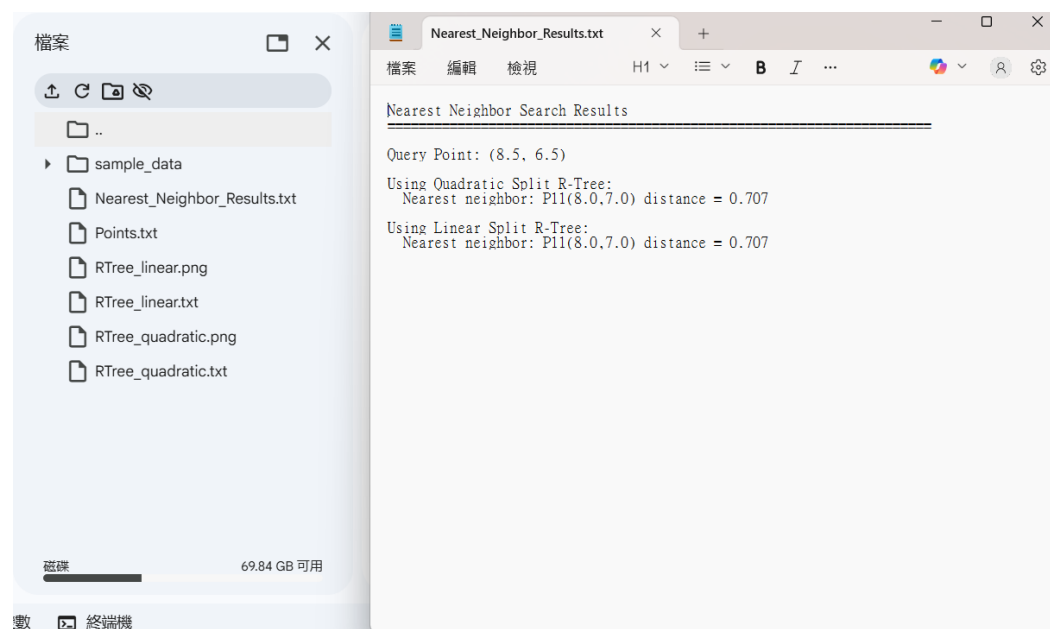
```

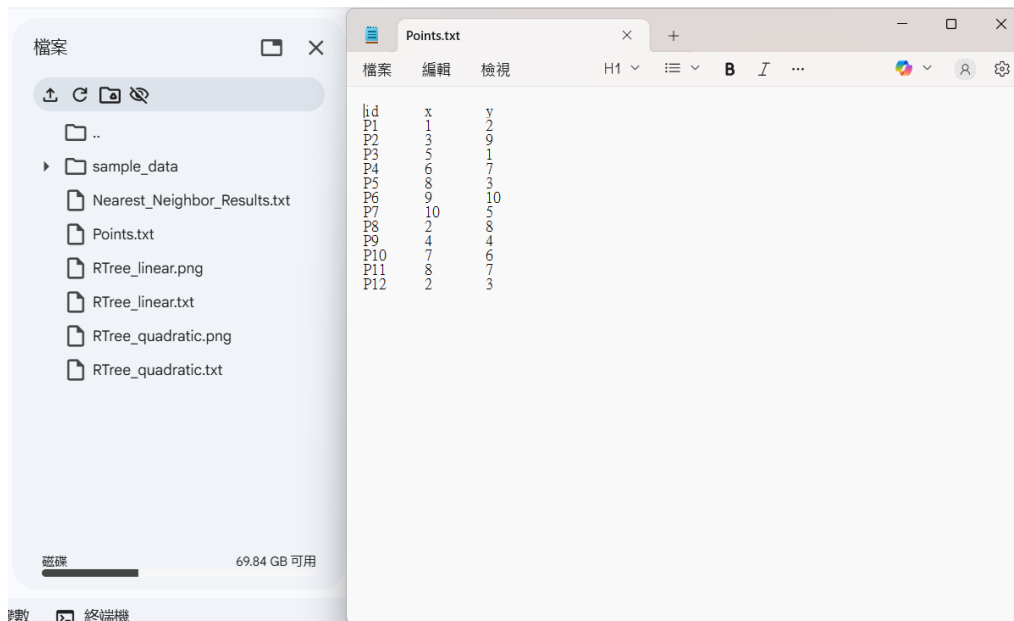
        if len(max_heap) < k or min_dist < -
max_heap[0][0]:
        heapq.heappush(min_heap, (min_dist,
next(self.counter), child))

    result = [(-dist, point) for dist, point in max_heap]
    result.sort()
    return result

```

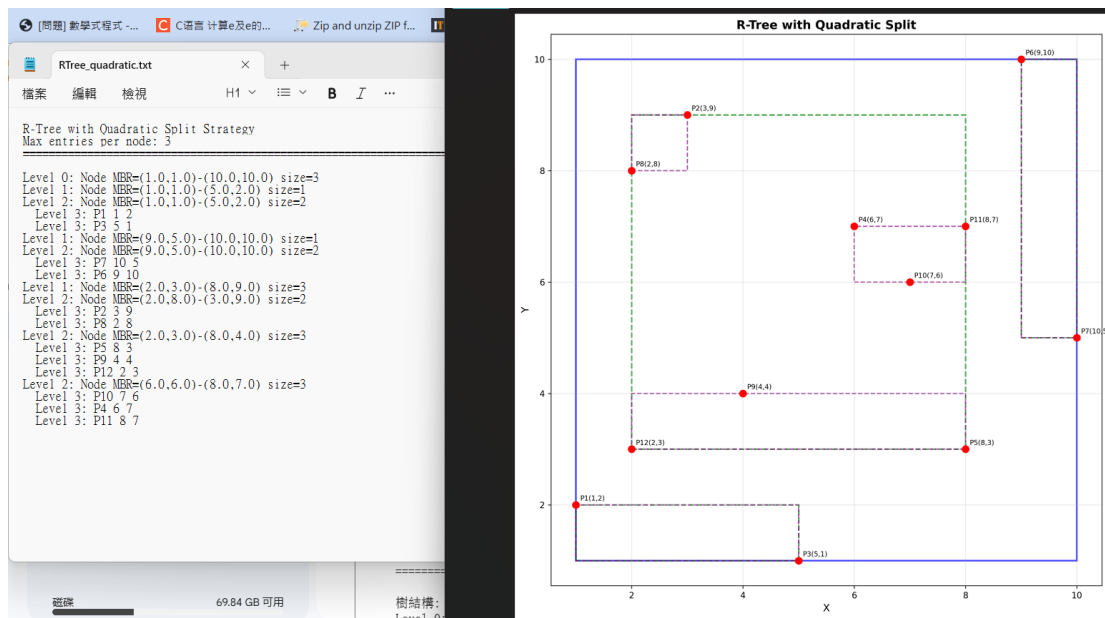
執行結果:



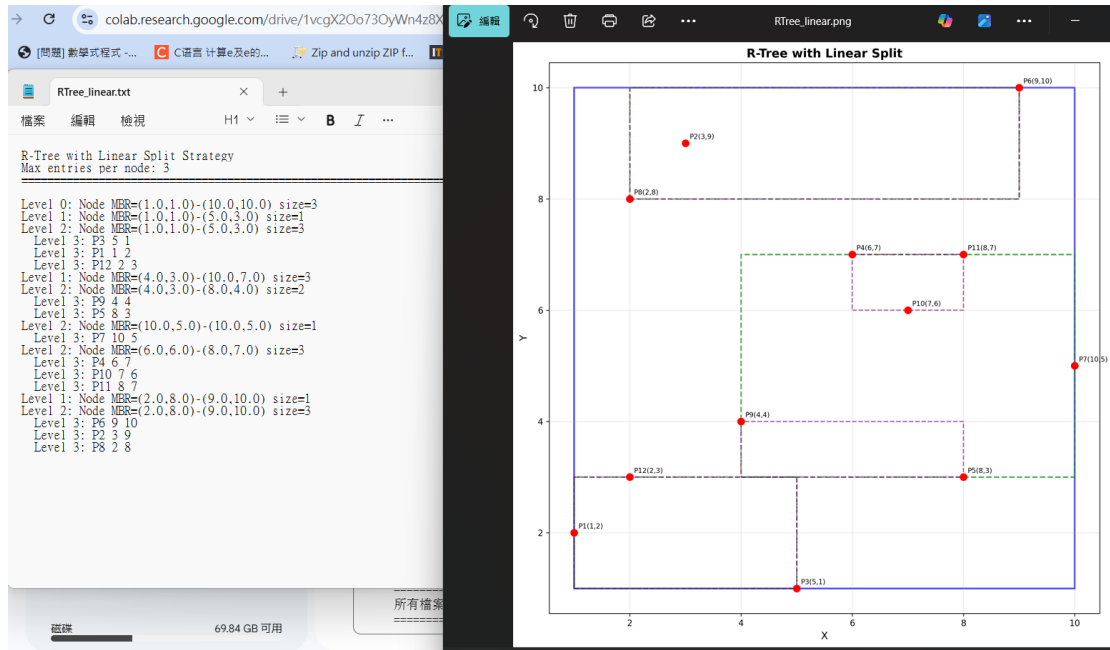


數 終端機

RTree_quadratic:



RTree_linear:



Linear split：比較簡單、快速的策略，概念是先找在 x 或 y 方向上距離最遠的兩個 entry 當種子，剩下的再依「加入哪一組增加面積比較少」去分配。

Quadratic split：會看所有成對的 entry，算「兩個合在一起浪費多少面積（waste）」，選浪費最大的那一對當種子，之後一樣把剩下的 entry 分配到兩組。

完整程式碼:

<https://colab.research.google.com/drive/1vcgX2Oo73OyWn4z8XMfqVtRl-xZlcysO?usp=sharing>